

Comparative Evaluation of Transfer Learning Strategies for AI-Generated Face Detection using EfficientNetB0, ResNet50, and MobileNetV2

1st Rismiyati*

Department of Computer Science and
Electronics
Universitas Gadjah Mada
Yogyakarta, Indonesia
rismiyati@mail.ugm.ac.id

2nd Herlina*

Department of Computer Science and
Electronics
Universitas Gadjah Mada
Yogyakarta, Indonesia
herlina@mail.ugm.ac.id

3rd I Gede Bintang Arya Budaya*

Department of Computer Science and
Electronics
Universitas Gadjah Mada
Yogyakarta, Indonesia
igedebintangaryabudaya@mail.ugm.ac.id

Abstract—Advances in Generative Adversarial Networks (GAN)-based image synthesis technology have triggered the urgency of developing robust digital forensic methods for detecting deepfakes. Although Transfer Learning techniques using Convolutional Neural Networks (CNNs) have become the standard, their general implementation is often limited to the use of standard head classification without in-depth exploration of the impact of final architecture modifications and training strategies on the model's generalization ability. This research aims to optimize AI-generated face image detection strategies through a systematic comparative study of three backbone architectures: EfficientNetB0, ResNet50, and MobileNetV2. The proposed method involves evaluating 12 experimental scenarios that include variations in classification head designs (comparing conventional Dense layers with Hybrid architectures based on Custom Conv2D 3x3 and 7x7 kernels) and analyzing the impact of a Fine-Tuning strategy (50% weight unveiling). The models were trained and tested using open dataset. The results show that EfficientNetB0 with a Hybrid Head configuration achieved the best performance with an accuracy of 93.35%, outperforming MobileNetV2 (91.50%) and ResNet50 (88.25%). Another key finding indicates that a Fine-Tuning strategy is crucial for drastically improving the performance of MobileNetV2 and ResNet50.

Keywords—deep learning, pre-trained model, fake image, fine-tuning model.

I. INTRODUCTION

The development of Artificial Intelligence (AI) has had a significant impact on human life, both positive and negative. Like a double-edged sword, AI, touted as capable of increasing human productivity, also poses significant problems [1]. This problematic phenomenon points to the unethical use of AI technology. One emerging issue is DeepFake, an AI-based generated image and video manipulation technique capable of producing synthetic content with near-real quality, making it difficult to distinguish from the original in specific contexts. This phenomenon has also occurred in Indonesia and is actually increasing. Several cases have involved public figures and political figures who have become the targets of DeepFake content experiments. DeepFake content creation is used for various purposes, ranging from cyberbullying and defamation to political propaganda. With the high use of social media, especially in Indonesia, the spread of such manipulative content can have profound impacts on individual reputations and social stability [2].

The technology behind DeepFake is heavily supported by the development of AI-based Generative Adversarial Networks (GANs), which can generate synthetic images with high levels of detail [3]. On the one hand, the goal of developing GANs is to serve positive purposes in design, education, and computer science [4]. However, their misuse poses significant threats, including disinformation, identity fraud, and privacy violations. Therefore, various research efforts have been conducted to address the negative aspects of DeepFake content, including research on detecting AI-generated images [5], [6]. Various methods have been developed to detect fake images, including deep learning-based approaches. Convolutional Neural Network (CNN) is one of the most effective deep learning architectures for extracting visual patterns that are not easily recognized by the human eye. However, using only standard CNN architectures poses challenges and limitations in detecting DeepFake.

This research aims to develop a deepfake detection system. This research adopts a comparative study approach by systematically evaluating three leading custom CNN MobileNetV2, EfficientNetB0, and ResNet50. The selection of these three models represents a spectrum of complexity and different architectural characteristics. First is EfficientNetB0, which offers a balance of performance through the compound scaling method [7]. Second is ResNet50, which is included as an industry standard known for its architectural depth for deep feature extraction [8]. While the third is MobileNetV2 [9], which was chosen for its efficiency through inverted residual blocks, which are ideal for light computation.

To ensure an objective and comprehensive evaluation, this study implemented a total of 12 identical experimental scenarios across the three backbone architectures. The experiments were designed to optimize the classification head design by comparing the effectiveness of a conventional Dense layer (using Dropout regularization variations of 0.3, 0.5, and 0.7) versus a Hybrid architecture integrating custom Convolutional layers (Custom Conv2D with 3x3 and 7x7 kernels) before classification [10]. Furthermore, this study explored the impact of training strategies by comparing a standard transfer learning method (frozen weights) against a fine-tuning strategy (50% weight unwinding) to gauge each model's adaptability to the deepfake domain [11]. All models were trained and tested using the public Real vs. Fake Face Detection dataset, which combines real face images from the

FFHQ dataset and synthetic faces generated by StyleGAN [12]. The performance of the three models was evaluated using accuracy, precision, recall, and F1-Score metrics. Through this systematic approach, this research is expected to provide empirical insights into the trade-off between computational efficiency and detection accuracy.

This research is structured as follows: Section I provides the background of the problem, the urgency of the study, and the purpose of the comparison between models. Section II discusses the literature study related to the state-of-the-art in image-based deepfake detection as well as the technical characteristics of the EfficientNetB0, ResNet50, and MobileNetV2 architectures. Next, Section III explains the methodological framework, starting with the preparation of the Real vs Fake Face dataset, hyperparameter configuration, and the implementation details of 12 test scenarios (Dense, Custom Conv2D, and Fine-Tuning variations). Section IV demonstrates the experimental results, displays performance visualizations, and analyzes the three model scenarios. This paper closes with Section V, which contains conclusions and suggestions for further research.

II. LITERATURE REVIEW

The advancement in Generative Adversarial Networks (GAN)-based image synthesis technology, such as StyleGAN [13], has presented significant challenges in distinguishing real faces from fakes. To address this, Convolutional Neural Networks (CNN)-based approaches with Transfer Learning techniques have become the state-of-the-art standard. Recent literature has extensively explored the use of architectures such as ResNet50 [8], which excels in feature depth, MobileNetV2 [9], which prioritizes computational efficiency through inverted residuals, and EfficientNetB0 [7], which offers balanced performance through compound scaling. These three architectures are often the primary choice for image classification tasks due to their robust feature extraction capabilities.

The three selected CNN architectures as a feature extractor have been widely validated across various domains, but empirical evidence suggests significant performance variability depending on data characteristics. Previous research on a skin disease classification study by [14] reported superior performance for EfficientNet-B0 with an accuracy of 99%, followed by MobileNetV2 (94%) and ResNet50 (93%). Conversely, in a study by [15] on soil type classification, the performance of similar architectures fluctuated, with EfficientNetB0 achieving an accuracy of 87.50% and MobileNetV2 at 78.57%. This significant difference in accuracy across domains indicates that the reliability of the backbone architecture is not absolute. Therefore, directly adopting a pre-trained model without adjusting the training strategy, such as head modification and fine-tuning for the specific deepfake domain, risks producing suboptimal performance. This underlies the urgency of this study to evaluate the three models through 12 systematic experimental scenarios.

III. METHODOLOGY

The diagram in Figure 1 illustrates the overall workflow for developing and evaluating an image-forgery detection model. The process begins with Input Acquisition, where raw images are collected. These images are compiled into a

Dataset that includes both authentic and manipulated images. Next, the dataset undergoes a Preprocessing stage, which involves standardizing image sizes, normalizing pixel values, and applying data augmentation techniques to improve data variability and model robustness.

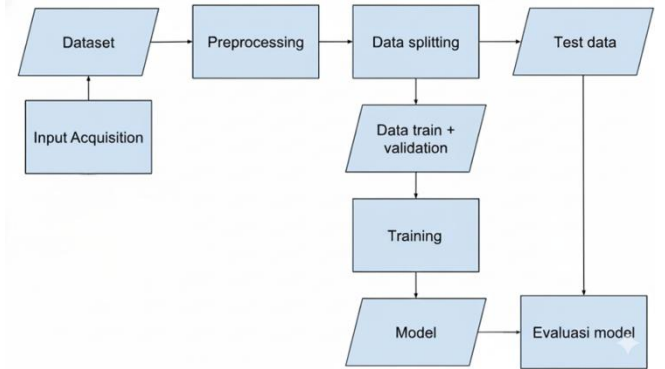


Figure 1. Research flow

Following preprocessing, the data are divided through a Data Splitting procedure into separate subsets, usually consisting of training, validation, and testing data. The Training and Validation Data subsets are then used in the Training phase, during which the model learns to distinguish between real and fake images. Once training is complete, the resulting Model is generated and subsequently evaluated. The Testing Data, which were not used during training, are fed into the model to assess its performance. Finally, the workflow concludes with the Model Evaluation step, where metrics such as accuracy, precision, recall, and other relevant indicators are analyzed to determine the effectiveness of the model.

A. Dataset

The experiment utilized a subset of the Kaggle 140K Real and Fake Faces Public Dataset [12]. This dataset consists of 140,000 images in total. For the purposes of this experiment, 22,000 images are taken and used. This dataset is then split into training, validation and test data. The breakdown of the dataset used in this experiment is as follows:

- 1) *Training data*: Consists of 20,000 images
- 2) *Validation data*: Consists of 2,000 images
- 3) *Test data*: Consists of 2,000 images

B. Preprocessing

The preprocessing stage is crucial for transforming raw image data into a standardized and suitable format for deep learning model training, ensuring consistency and optimal performance. The dataset used in this study consists of 22,000 images of real and fake faces. The preprocessing pipeline involved several key steps:

- 1) *Image Resizing and Normalization*: All images were resized to a uniform dimension to meet the input requirements of the selected models. Subsequently, pixel values originally ranging from 0–255 were normalized to a 0.0–1.0 range to standardize the data distribution. This normalization is essential for ensuring stable and faster model convergence.
- 2) *Data Augmentation*: To prevent overfitting and enhance the models' generalization capability, data augmentation techniques were applied exclusively

to the training dataset (20,000 images). These techniques included random rotations, horizontal flips, and minor adjustments to zoom and brightness. By artificially increasing the diversity of training data, the effective size and robustness of the dataset were improved.

- 3) **Label Encoding:** The categorical class labels ('Real' and 'Fake') were converted into a binary numerical format (0 or 1). This conversion ensures compatibility with the Binary Cross-Entropy loss function used in the model training phase.

C. Training

The preprocessed training data is fed into the proposed model as shown in Figure 2. The training starts with an input image of size 224x224x3 (height, width, color channels), which is passed through a pre-trained deep learning base model that processes the image features up to, but not including, the global average pooling or flattening layer. The extracted features are then fed into a 2D convolutional layer (conv2D) with parameters dependent on batch size and the number of filters. The output from this convolutional layer is flattened into a 1D vector, followed by a dense (fully connected) layer, which finally passes through a sigmoid activation function to produce the final output, typically representing a probability score for binary classification.

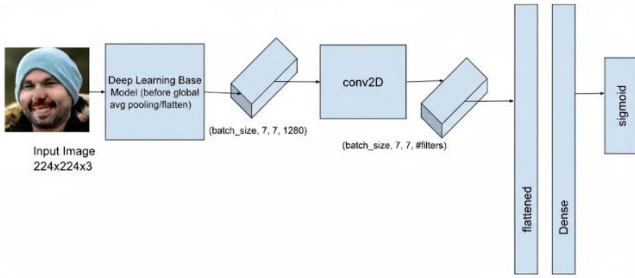


Figure 2. Proposed model

The base models used in this experiment are EfficientNetB0, ResNet50 and MobileNetV2.

- 1) **EFFICIENTNETB0:** EfficientNet scales convolutional neural networks by uniformly expanding depth, width, and input resolution using a compound scaling strategy. Instead of arbitrarily increasing one dimension, EfficientNet applies fixed scaling coefficients that allow the network to grow in a more structured and efficient way. The architecture relies on MBConv blocks with squeeze-and-excitation modules. This modol help the model capture richer feature representations while keeping the computational cost manageable. This combination of balanced scaling and optimized building blocks allows EfficientNet to achieve strong accuracy with significantly fewer parameters, making it a practical choice for both high-performance systems and more constrained environments [16].
- 2) **RESNET50:** ResNet50 is one of deep convolutional network architecture which use skip connection to overcome vanishing gradient problem. Despite its

large number of layers, this architecture can overcome the vanishing gradient problem due to the skip connection. The structure of residual block is shown in Figure 3 [17]. The Residual blocks help prevent vanishing gradients because the skip connection allows the training signal to flow directly back to earlier layers. This direct path keeps the gradient from shrinking too much, making it easier for very deep networks to learn.

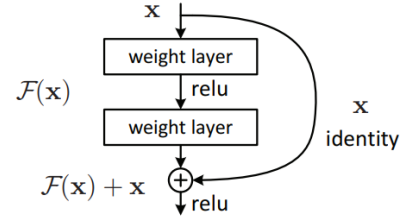


Figure 3. Residual block in ResNet50 [8]

- 3) **MOBILENETV2:** This model is a lightweight, efficiency-oriented architecture. As shown in Figure 4, MobileNet replaces the standard 3x3 convolution with a depthwise separable structure. In this design, each channel is filtered separately through a depthwise convolution, and the results are then combined using a 1x1 pointwise convolution. Both steps include batch normalization and a ReLU activation to ensure stable and efficient training. By breaking the operation into these two simpler stages, MobileNet greatly reduces the number of parameters and computational cost, making it well suited for deployment on mobile and other resource-limited devices [9].

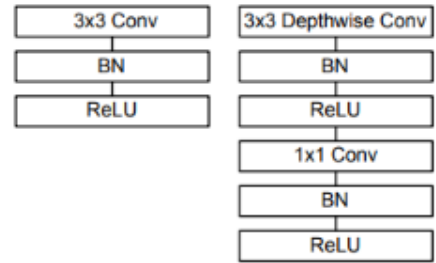


Figure 4. Depthwise separable convolution

D. Evaluation

The resulting model is evaluated by using common evaluation metrics for image classification. In this experiment, precision, recall, accuracy and F1-score of the model is evaluated and compared to determine the best model.

E. Experiment Scenarios

To determine the best model, several experiment scenarios were performed. The scenarios are as follows:

- 1) **Determine the structure of dense layer:** This scenario is performed with ResNet50 base layer, to determine the number of neurons in dense layer and the dropout used. The result of this scenario is used for the subsequent scenarios. In total 6 combination is performed, as a combination of varying neuron number (256,512) and also drop out of (0.3,0.5,0.7)

- 2) *Determine the structure of convolution layer:* The best model for scenario 1 is used for scenario 2. Additional convolution layer is added after the base model output. In total, 4 combination is used in this scenario, result of combination of number of kernel (256,512) and also kernel size ([3,3], [7,7]).
- 3) *Determine the impact of fine tune to the model:* This scenario is performed to determine the impact of fine tuning to the model. In this scenario, several models are trained with 50% fine tuning performed in base model.

IV. RESULT AND DISCUSSION

A. Result of RESNET50

Experiment results to determine the impact of neuron number and dropout in dense layer is shown in Table 1. Based on the results, the model's performance depends on applying Fine-Tuning, in this case at the 50% level. Even the simplest fine-tuned model (model 11), which reached 84.1% accuracy, performed better than the strongest baseline without Fine-Tuning, which achieved 78.0%. The models in the noFT group consistently showed lower performance overall, ranging from 61.85% to 78.0%, although adding an extra convolutional layer still provided a modest improvement compared to using only dense layers. The best performance was achieved when 50% Fine-Tuning was combined with an additional 3x3 convolution block. This configuration, Model 12 (FT 50% + Conv256 3x3 + Dense 256 DO 0.7), produced the highest results across all metrics, with an accuracy of 88.25% and an F1-Score of 90.27%.

TABLE I. EXPERIMENT RESULT ON RESNET50 FOR VARIOUS SCENARIOS

No	Model Name	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)
1	R_noFT_D256_DO0.3	68.25	68.53	67.50	68
2	R_noFT_D256_DO0.5	66.80	67.07	66.00	67
3	R_noFT_D256_DO0.7	68.90	69.09	68.40	69
4	R_noFT_D512_DO0.3	68.75	67.28	73.00	70
5	R_noFT_D512_DO0.5	61.85	57.56	90.20	70
6	R_noFT_D512_DO0.7	67.90	68.53	66.20	68
7	R_noFT_C256_3x3_D256_DO0.7	72.8	73.26	71.8	72.5
8	R_noFT_C256_7x7_D256_DO0.7	78.0	78.9	76.3	77.6
9	R_noFT_C512_3x3_D256_DO0.7	73.75	74.3	72.6	73.4
10	R_noFT_C512_7x7_D256_DO0.7	74.3	85.9	58.1	69.33
11	R_FT50_D256_DO0.7	84.5	85.2	84.1	84.6
12	R_FT50_C256_3x3_D256_DO0.7	90.05	92.4	88.25	90.27
13	R_FT50_C256_7x7_D256_DO0.7	87.9	88.6	88.2	88.0

The accuracy and loss during training and validation is shown in Figure 5 and Figure 6. Although all models managed to converge, with Training Loss steadily decreasing and Training Accuracy rising, each one displayed some level of overfitting, as seen from the consistent gap between the training and validation curves. Among them, the FT 50% + Conv2563x3 + Dense 256 DO 0.7 model demonstrated the

strongest generalization, achieving both the lowest Validation Loss and the highest Validation Accuracy at the end of training. On the other hand, models without fine tune showed the weakest outcomes. The best scenario for ResNet50 is achieved where the confusion matrix for the scenario is shown in Figure 7.

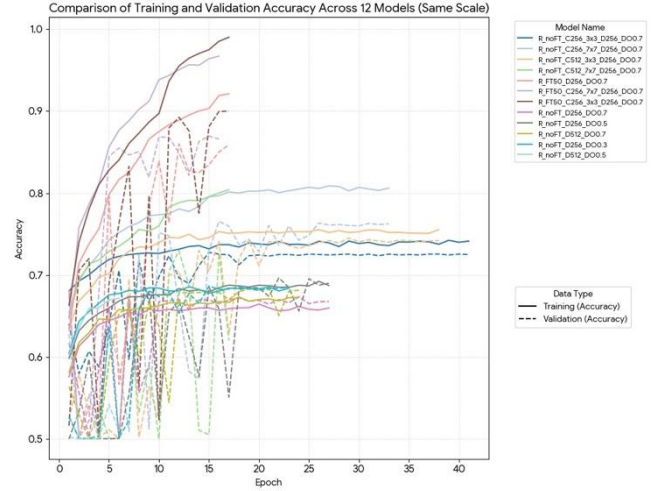


Figure 5. Comparison of training and validation accuracy for various scenarios for ResNet50

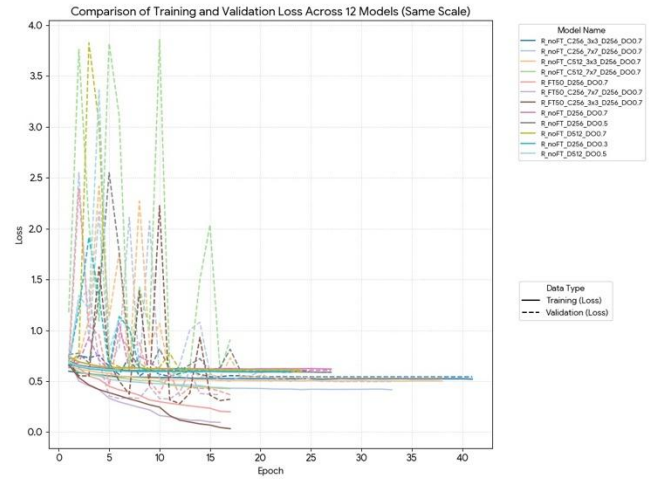


Figure 6. Comparison of training and validation loss for various scenarios for ResNet50

True	0	877	123
	1	76	924
	Predicted	0	1

Figure 7. Confusion matrix for the Resnet50 best model

B. Result of EfficientNetB0

Experimental results on various EfficientNetB0 modification scenarios in Table 2 show high performance stability with consistent accuracy in the range of 0.925 to 0.935, indicating that the integration of the convolution block in the classification head is able to maintain the model's generalization ability. Among all tested configurations, the use of a convolution kernel size of 3x3 (with both 256 and 512 filters) proved to be the most superior, recording the highest F1-scores of 0.9338 and 0.9350, respectively. This superiority is also supported by better training efficiency with shorter convergence times, confirming that small kernels are more effective in enriching local spatial feature representation without excessively increasing the computational burden.

TABLE II. EXPERIMENT RESULT ON EFFICIENTNETB0 FOR VARIOUS SCENARIOS.

No	Model Name	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)
1	E_noFT_C256_3x3_D256_DO0.7	93.35	91.40	95.70	93.50
2	E_noFT_C256_7x7_D256_DO0.7	92.95	93.08	92.80	92.94
3	E_noFT_C512_3x3_D256_DO0.7	93.35	92.96	93.80	93.38
4	E_noFT_C512_7x7_D256_DO0.7	92.55	91.51	93.80	92.64
5	E_FT50_C256_3x3_D256_DO0.7	91.65	91.36	92.00	91.68
6	E_FT50_C256_7x7_D256_DO0.7	92.40	92.57	92.20	92.38
7	E_FT50_C512_3x3_D256_DO0.7	91.75	89.95	94.00	91.93
8	E_FT50_C512_7x7_D256_DO0.7	92.50	92.33	92.70	92.51

In contrast, applying a kernel with a wider receptive field (7x7) did not provide a significant performance improvement, even experiencing a decrease in the F1-score to the range of 92.64%–92.94%, accompanied by the need for longer epochs to reach optimal conditions. This suggests that the deep features extracted by the EfficientNetB0 backbone no longer require additional receptive field expansion in the final classification stage. Although the 50% feature reduction scenario before convolution 7x7 was able to improve stability with an F1 score reaching 0.9310, its overall performance was still unable to exceed the effectiveness and efficiency of the 3x3 kernel configuration. Figures 8 and 9 show the training and validation accuracy and loss, while Figure 10 shows the confusion matrix for the best model.

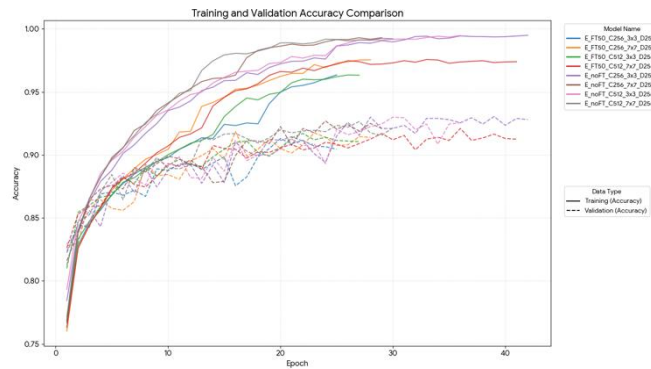


Figure 8. Comparison of training and validation accuracy for various scenarios for EfficientNetB0

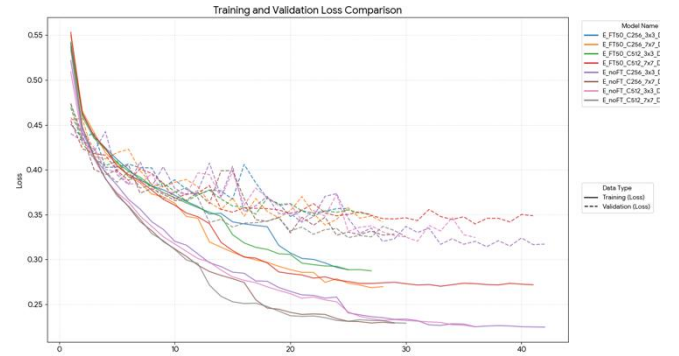


Figure 9. Comparison of training and validation loss for various scenarios for EfficientNetB0

Overall, this analysis concludes that the combination of the EfficientNetB0 architecture with 3x3 convolution blocks (using 256 or 512 filters) is the most optimal approach. This configuration offers the best balance between high accuracy, model stability, and training efficiency compared to other scenarios. Therefore, the hybrid architecture with 3x3 kernels is recommended as the primary configuration in this study's model development, as it has been shown to maximize detection performance without significantly increasing model complexity.

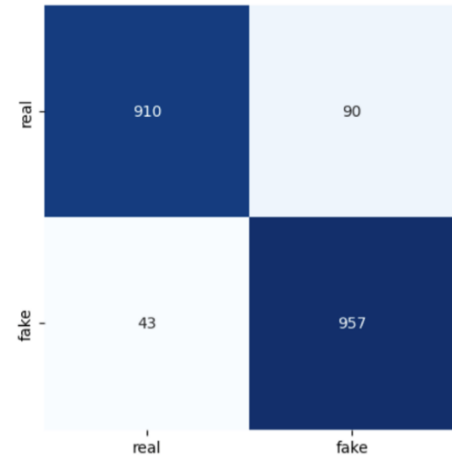


Figure 10. Confusion matrix of EfficientNetB0

C. Result of MobileNetV2

The accuracy and loss during training and validation is shown in Figure 11 and Figure 12. Based on the evaluation results of 13 scenarios using the MobileNetV2 backbone, a significant pattern of performance improvement was observed, influenced by the classification head design and training strategy. In general, models using the Frozen Weights strategy (Scenarios 1-10) achieved stagnant accuracy, ranging from 81.45% to 85.35%. In this group, the modified Hybrid architecture using additional convolutional layers (Scenarios 7-10) proved to offer a slight advantage over the standard Dense architecture. The M_noFT_C512_7x7_D256_DO0.7 model (Scenario 10) recorded the highest accuracy in the frozen group at 85.35%. This indicates that the addition of local spatial extraction features (Custom Conv2D) helps MobileNetV2 better capture manipulation artifacts than relying solely on direct Global Average Pooling.

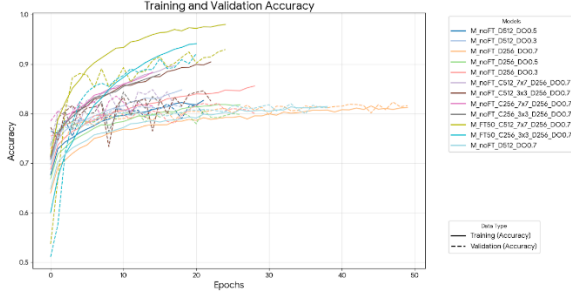


Figure 11. Comparison of training and validation accuracy for various scenarios for MobileNetV2



Figure 12. Comparison of training and validation accuracy for various scenarios for MobileNetV2

The most drastic performance improvement occurred when the Fine-Tuning strategy was applied (unweighting the last 50% of the layers) as shows in Table 3. The M_FT50_C512_7x7_D256_DO0.7 (Scenario 12) model emerged as the best performing model with 91.50% Accuracy, 92.19% Precision, and 91.47% F1-Score. The $\pm 6\%$ improvement compared to the best frozen model indicates that the features learned by MobileNetV2 on the dataset need to be deeply retrained to be sensitive to synthetic facial patterns. The combination of Fine-Tuning and the Hybrid Head architecture (Conv 7x7) in Scenario 12 proved to be the most optimal configuration for this test dataset. The confusion matrix shows in Figure 12.



Figure 13. Confusion matrix of MobileNetV2

TABLE III. EXPERIMENT RESULT ON MOBILENETV2 FOR VARIOUS SCENARIOS

No	Model Name	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)
1	M_noFT_D256_DO0.3	83.00	83.21	83.00	82.97
2	M_noFT_D256_DO0.5	81.95	81.95	81.95	81.95
3	M_noFT_D256_DO0.7	82.45	82.45	82.45	82.45
4	M_noFT_D512_DO0.3	81.45	81.47	81.45	81.45
5	M_noFT_D512_DO0.5	81.95	81.97	81.95	81.95
6	M_noFT_D512_DO0.7	81.45	81.49	81.45	81.44
7	M_noFT_C256_3x3_D256_DO0.7	83.5	83.65	83.50	83.48
8	M_noFT_C512_3x3_D256_DO0.7	84.05	84.11	84.05	84.04
9	M_noFT_C256_7x7_D256_DO0.7	83.5	83.69	83.50	83.48
10	M_noFT_C512_7x7_D256_DO0.7	85.35	85.55	85.35	85.33
11	M_FT50_C256_3x3_D256_DO0.7	90.85	91.06	90.85	90.84
12	M_FT50_C512_7x7_D256_DO0.7	91.50	92.19	91.50	91.47

D. Discussion

Based on a series of experiments that have been carried out in stages on the MobileNetV2, ResNet50, and EfficientNetB0 architectures, this section discusses the implications of each stage of architectural modification and compares the best performance of the three models. TABLE IV shows each best model scenarios.

1) Determining the Dense Layer Structure (Baseline):

In the initial phase, the evaluation focused on establishing a performance baseline using a standard classification layer (Dense Layer). Based on experimental results for six combinations (256/512 neuron variations and Dropout 0.3/0.5/0.7), it was found that using the Dense Layer alone tended to produce limited performance, particularly on the ResNet50 architecture, which only achieved accuracy in the range of 60-68%. This indicates that the global features extracted by the backbone using Global Average Pooling still require further filtering to separate subtle the data artifacts. The configuration with Dropout 0.7 consistently provided the best stability in this phase, as the high level of regularization forces the model to learn more robust features and prevents overfitting to the training data. Therefore, this configuration was chosen as the foundation for the next phase.

2) Determining the Convolution Layer Structure:

The addition of a custom convolution block (Custom Conv2D) after the backbone proved to be a crucial step in improving detection capabilities. Experiments showed that integrating convolution kernels, particularly 3x3 kernels, provided a significant positive impact compared to using only a Dense Layer. Specifically, small kernels (3x3) proved more effective than large kernels (7x7) in most scenarios, especially on EfficientNetB0 and ResNet50. This is logical considering that images (especially those generated by GANs) often leave traces of artifacts at the pixel level or fine textures

(high-frequency noise). The 3x3 kernel is able to capture these local anomalies in greater detail. Conversely, the 7x7 kernel, with its wider receptive field, tends to obscure the details of these micro-artifacts, which is evident in slightly lower or stagnant performance in some scenarios. Therefore, the Hybrid architecture with a 3x3 kernel is the most recommended configuration for advanced spatial feature extraction.

- 3) *Analysis of the Impact of Fine-Tuning:* The fine-tuning strategy (unloading 50% of the weights in the backbone layers) proved to be a major game-changer in the model performance gains, particularly for MobileNetV2 and ResNet50. On ResNet50, this model experienced the most drastic performance improvement, jumping from ~74% accuracy to 88.25% after fine-tuning. This indicates that the initial weights of ResNet50 were less relevant for the dataset in this case, and fine-tuning allowed it to fully adapt to the new domain. On MobileNetV2, similar to ResNet, performance increased significantly from ~85% to 91.50%. On EfficientNetB0, interestingly, this model already demonstrated superior performance (F1-Score 93.50%) even with high stability across convolution variations, indicating that EfficientNet's baseline features were already very rich and representative. This confirms that fine-tuning is essential for models with older or less parameter-efficient architectures, but modern models like EfficientNet already have very robust features from the start.

TABLE IV. COMPARISON OF EACH BEST MODEL SCENARIOS

No	Model Base	Configuration	Accuracy (%)
1	EfficientNetB0	R_FT50_C256_3x3_D256_D00.7	93.35
2	ResNet50	E_noFT_C256_3x3_D256_D00.7	90.05
3	MobileNetV2	M_FT50_C512_7x7_D256_D00.7	91.50

V. CONCLUSION

This study concludes that the EfficientNetB0 architecture integrated with a custom architecture-based classification head (3x3 custom convolutional layers with 256 filters) is the most superior configuration for synthetic face detection, achieving the best performance with an F1-Score of 93.50%. This finding confirms that adding a small kernel (3x3) convolutional layer at the final stage of the network is much more effective in capturing subtle spatial artifacts typical of deepfakes compared to using standard dense layers or large kernels (7x7). Furthermore, while the Fine-Tuning strategy proved to be a crucial factor in boosting the performance of MobileNetV2 and ResNet50 to be competitive, EfficientNetB0 demonstrated more robust and efficient feature stability without the need for aggressive weight tuning, making it the most recommended architectural choice as it offers an optimal balance between high accuracy and model robustness.

REFERENCES

- [1] F. Federspiel, R. Mitchell, A. Asokan, C. Umana, and D. McCoy, "Threats by artificial intelligence to human health and human existence," *BMJ Glob Health*, vol. 8, no. 5, May 2023, doi: 10.1136/bmjgh-2022-010435.
- [2] "(PDF) Exploding AI-Generated Deepfakes and Misinformation: A Threat to Global Concern in the 21st Century," *ResearchGate*, Aug. 2025, doi: 10.32388/DPLE2L.
- [3] A. Creswell, T. White, V. Dumoulin, K. Arulkumaran, B. Sengupta, and A. A. Bharath, "Generative Adversarial Networks: An Overview," *IEEE Signal Processing Magazine*, vol. 35, no. 1, pp. 53–65, Jan. 2018, doi: 10.1109/MSP.2017.2765202.
- [4] N. Waqas, S. I. Safie, K. A. Kadir, S. Khan, and M. H. Kaka Khel, "DEEPFAKE Image Synthesis for Data Augmentation," *IEEE Access*, vol. 10, pp. 80847–80857, 2022, doi: 10.1109/ACCESS.2022.3193668.
- [5] C. Yang, L. Ding, Y. Chen, and H. Li, "Defending against GAN-based DeepFake Attacks via Transformation-aware Adversarial Faces," in *2021 International Joint Conference on Neural Networks (IJCNN)*, July 2021, pp. 1–8. doi: 10.1109/IJCNN52387.2021.9533868.
- [6] Preeti, M. Kumar, and H. K. Sharma, "A GAN-Based Model of Deepfake Detection in Social Media," *Procedia Computer Science*, vol. 218, pp. 2153–2162, Jan. 2023, doi: 10.1016/j.procs.2023.01.191.
- [7] K. K. K. S. A. K. J. and C. B., "Enhancing Skin Cancer Classification Using Efficient Net B0-B7 through Convolutional Neural Networks and Transfer Learning with Patient-Specific Data," *Asian Pac J Cancer Prev*, vol. 25, no. 5, pp. 1795–1802, 2024, doi: 10.31557/APJCP.2024.25.5.1795.
- [8] B. Koonce, "ResNet 50," in *Convolutional Neural Networks with Swift for Tensorflow: Image Recognition and Dataset Categorization*, B. Koonce, Ed., Berkeley, CA: Apress, 2021, pp. 63–72. doi: 10.1007/978-1-4842-6168-2_6.
- [9] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," in *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, Salt Lake City, UT: IEEE, June 2018, pp. 4510–4520. doi: 10.1109/CVPR.2018.00474.
- [10] I. Salehin and D.-K. Kang, "A Review on Dropout Regularization Approaches for Deep Neural Networks within the Scholarly Domain," *Electronics*, vol. 12, no. 14, p. 3106, Jan. 2023, doi: 10.3390/electronics12143106.
- [11] A. Karathanasis, J. Violas, and I. Kompatsiaris, "A Comparative Analysis of Compression and Transfer Learning Techniques in DeepFake Detection Models," *Mathematics* (2227-7390), vol. 13, no. 5, p. 887, Mar. 2025, doi: 10.3390/math13050887.
- [12] "140k Real and Fake Faces." Accessed: Dec. 02, 2025. [Online]. Available: <https://www.kaggle.com/datasets/xhlulu/140k-real-and-fake-faces>
- [13] A. h. Bermanno *et al.*, "State-of-the-Art in the Architecture, Methods and Applications of StyleGAN," *Computer Graphics Forum*, vol. 41, no. 2, pp. 591–611, 2022, doi: 10.1111/cgf.14503.
- [14] M. Y. Shams *et al.*, "Skin Disease Classification: A Comparison of ResNet50, MobileNet, and Efficient-B0," *Journal of Current Multidisciplinary Research*, vol. 1, no. 1, pp. 1–7, Jan. 2025, doi: 10.21608/jcmr.2025.327880.1002.
- [15] A. Joshi, R. Ragel, M. Mahmud, and S. Kartik, *ICT: Applications and Social Interfaces: Proceedings of ICTCS 2024, Volume 2*. Springer Nature, 2025.
- [16] B. Koonce, "EfficientNet," in *Convolutional Neural Networks with Swift for Tensorflow: Image Recognition and Dataset Categorization*, B. Koonce, Ed., Berkeley, CA: Apress, 2021, pp. 109–123. doi: 10.1007/978-1-4842-6168-2_10.
- [17] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA: IEEE, June 2016, pp. 770–778. doi: 10.1109/CVPR.2016.90.